MLDS Exploratory Data Analysis & Visualisation

Week 6 - Spatial Data

James S Martin

Spring Term, 2024

- Visualise your data
- Explore distributional properties
- Identify and fit trend components
- Examine spatial dependence structures
    - The semivariogram
    - The empirical semivariogram
    - An alternative empirical approach
- Using ggplot2 to create a choropleth map
- References

In this notebook, we will take a look at some key aspects of EDA for spatial data – this is often referred to as Exploratory Spatial Data Analysis (ESDA).

For the majority of this lab, we're going to focus on numeric data variables with associated spatial coordinates. There are alternative forms of spatial data – for instance, where the spatial information is specified by a nominal variable, such as the name of a geographical region (e.g. consider the `USArrests` dataset in R, which specifies crime in different US states). Most of the methods that we will consider will benefit from quantitative specification of the location information, and so we will focus on data that is associated with point locations.

A significant subfield of ESDA involves exploration of the point pattern formed by the location coordinates; that is, we could be interested in the locations themselves as a subject of our analysis. We will avoid these analyses in this notebook; we will consider the location information to be secondary to – though still crucially important to the analysis of – other observed data variables in the dataset. With this setup, we are able to treat the observed data variable of interest as an observation of a random field – that is, of a stochastic process with an index set that is two-dimensional. The important point to note here is that, as with time series data, spatial datasets usually comprise a single observation; they cannot be considered as a random sample containing many independent observations.

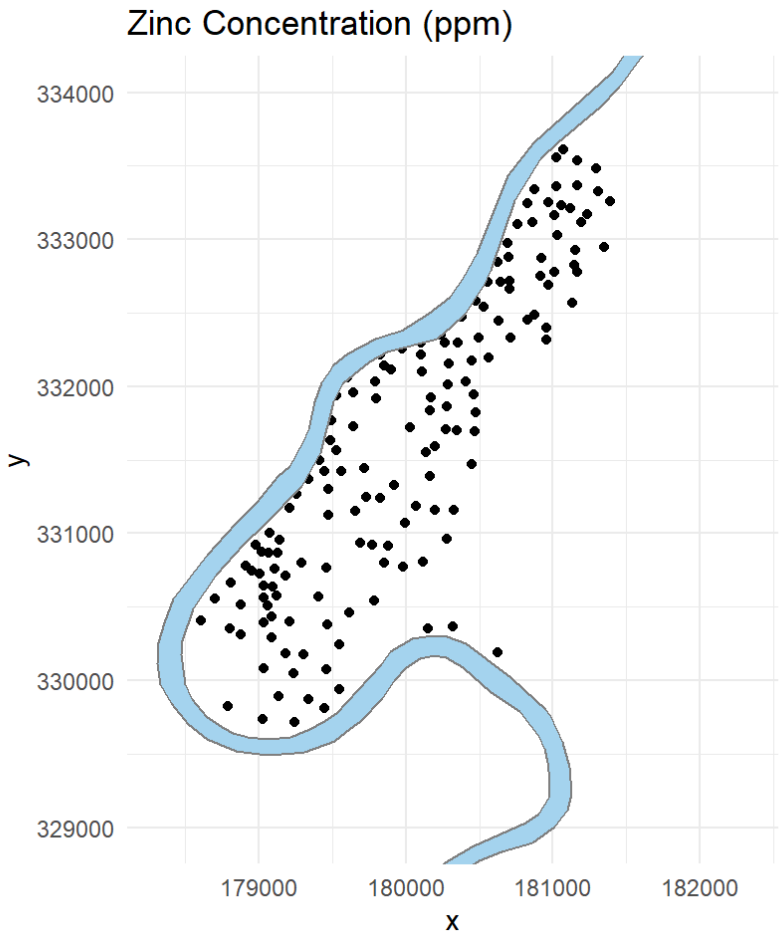
Throughout this notebook, we will consider the Meuse river dataset. This is a spatial dataset containing location coordinates and topsoil heavy metal concentrations (along with a number of soil and landscape variables at the observation locations), collected in a flood plain of the river Meuse, near the village of Stein (in the Netherlands). We will reduce our focus to the univariate analysis of `meuse$zinc` ; that is, we will treat this data as a realisation of a *univariate* random field. Some of the methods below can be extended (with a little work) to the multivariate case, where each spatial point is associated with multiple data variables.

Visualise your data

The first step of any exploratory data analysis is to visualise the raw data, in order to pick up on any obvious trends or patterns. ESDA is no different! We map the spatial component of our data in two-

dimensional space; for geographical data (i.e. data whose spatial coordinates correspond to points on the surface of the Earth), this can often be overlaid with suitable geographical features.

```
data(meuse)
data(meuse.riv)

ggplot(data=meuse, aes(x, y)) + geom_point() +
  geom_polygon(data = data.frame(x = meuse.riv[,1],y = meuse.riv[,2]),
               aes(x = x,y = y),
               fill = "lightskyblue2",
               colour= "grey50") +
  ggtitle("Zinc Concentration (ppm)") + coord_equal(ylim=c(329000,334000)) + theme
_bw() +
  scale_size(range = c(0.5, 3.5)) +   theme_minimal()
```



When it comes to representing the value of our covariate on our map, we have two broad options:

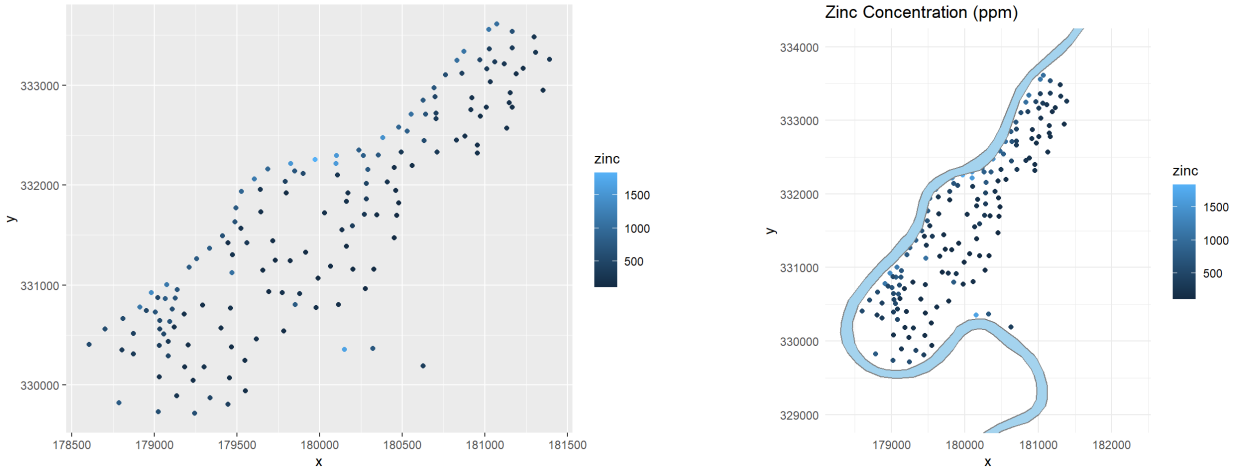
- we can turn to three-dimensional visualisations, adding the value of our variable on a third (typically vertical) axis; or
- we can encode the value of the observed data in some aesthetic property of the visualisation.

The second option is slightly more appealing, as it offers a little more flexibility in presentation.

Exercises

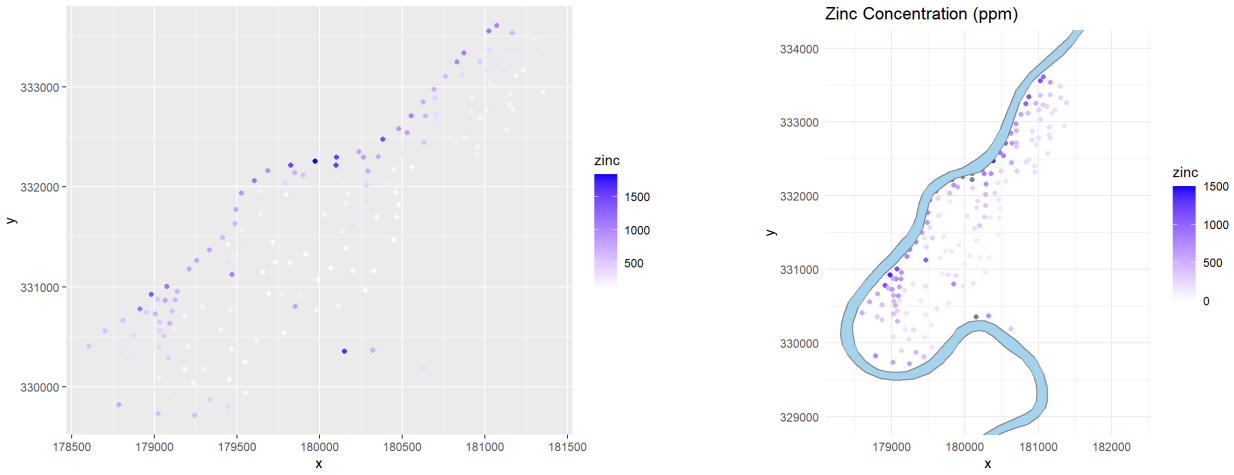
1. Use colour to encode the topsoil concentration of zinc in the `meuse` dataset. Try some alternative aesthetic mappings also. Which do you find most useful?
2. What trends can we pick up from this initial mapping of the data?

```
# Q1 - create a ggplot using the colour aesthetic to encode the zinc variable
# the following isolates the command that encodes the required aesthetic mapping
zincplot <- ggplot(data=meuse, aes(x, y)) + geom_point(aes(colour=zinc))

# of course we can also add in the extra details as in the plot provided above
zincplot2 <- zincplot +
  geom_polygon(data = data.frame(x = meuse.riv[,1],y = meuse.riv[,2]),
               aes(x = x,y = y),
               fill = "lightskyblue2",
               colour= "grey50") +
  ggtitle("Zinc Concentration (ppm)") + coord_equal(ylim=c(329000,334000)) + theme
_bw() +
  scale_size(range = c(0.5, 3.5)) +   theme_minimal()

zincplot
zincplot2
```



From these initial plots of the topsoil zinc concentration, we can see that concentration levels generally seem to be higher, closer to the banks of the river. This is perhaps more apparent using a different colour scale, as below:

```
zincplot + scale_colour_gradient(low="white", high="blue")
zincplot2 + scale_colour_gradient(low="white", high="blue", limits=c(0,1500))
```



One important aspect of the data that can be highlighted by simple maps such as the above, is regions of potential clustering. This may be of interest in itself (particularly in point pattern analysis), but it may also

be worth taking into account when interpreting distributional properties of the data. If the coordinates are not regularly spaced, or are randomly, but not uniformly spaced, then there may be *preferential sampling* effects; it may be that the sampled locations themselves are dependent on the variable being measured. In this case, certain values of the variable may be over-represented in the sample, and we would not be able to use the sample as a representation of the overall population, without first accounting for the spatial dependence.

Explore distributional properties

Once we have established the basic spatial patterns in the data, it is worth investigating the distributional properties of the data variable of interest, independently of its spatial component.

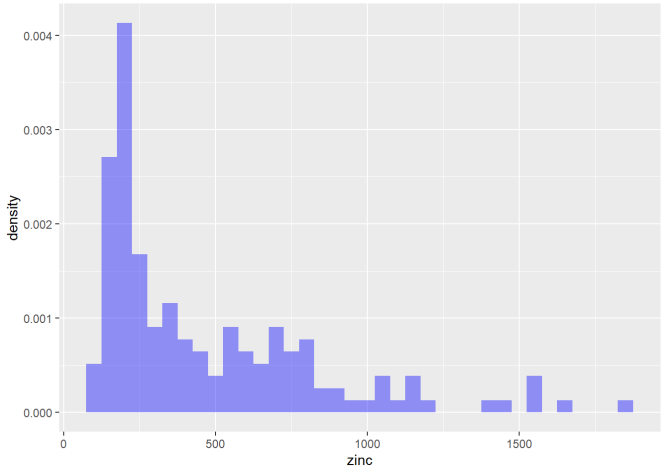
We therefore start with a look at the univariate distribution of our data variable. This can be achieved straightforwardly using EDA tools we've considered already in this module, such as histograms and Q-Q plots.

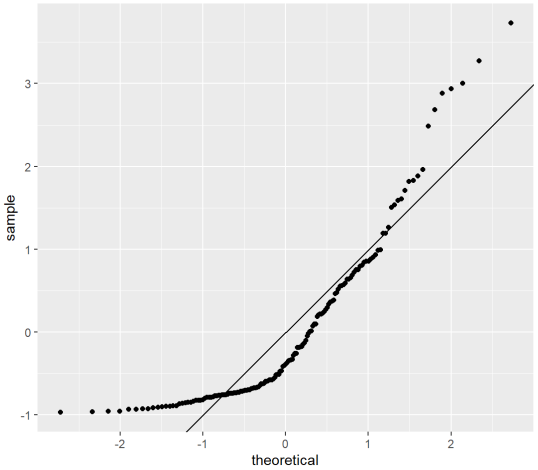
Exercises

3. Use a histogram and Q-Q plot to examine the distribution of zinc levels from the Meuse river dataset. Does the variable seem (approximately) Gaussian distributed?

We use ggplot to construct both the histogram and the Q-Q plot. For the latter, we standardise the values in `meuse$zinc` and compare against a sample from the standard Gaussian distribution, i.e. $N(0, 1)$.

```
# add a new variable to the data frame, containing the standardised zinc concentra
tions
meuse <- meuse %>%
  dplyr::mutate(stdzinc = (zinc-mean(zinc))/sd(zinc))

zinc_hist <- ggplot(data=meuse) +
  geom_histogram(mapping=aes(x=zinc, y=after_stat(density)), fill='blue', alpha=
0.4, binwidth=50)

zinc_qqnorm <- ggplot(data=meuse) +
  geom_qq(mapping=aes(sample=stdzinc)) +
  geom_abline(slope=1) + coord_fixed()

zinc_hist
zinc_qqnorm
```





From both the histogram and the Q-Q plot, we can immediately see that the distribution of this variable is

not Gaussian; the data appears to be positively-skewed. Taking the context of the data into account, we can also note that `meuse$zinc` takes non-negative values, and so for this reason also, a Gaussian distribution would not be an ideal modelling choice (as it is allocates non-zero probability density to all the real numbers).

The above examination of the data fails to take into account any effects of preferential sampling. There appears (at least at first glance) to be a nontrivial degree of clustering in the spatial component of the data. We could assess this more formally using measures of spatial dependence, in order to establish whether the clustering is, in some sense, significant. For now, let us assume it to be significant enough to potentially cause preferential sampling.

In order to examine any potential effects of preferential sampling, we apply a technique known as *cell declustering*. Broadly, our aim is to divide our spatial region into regular rectangular cells (i.e. of the same size), and to weight each datapoint according to the number of points that lie in its cell (including itself). The weight assigned to a given datapoint is calculated as follows:

- each cell that contains data is allocated the same weight (such that these cell weights sum to 1); and
- each cell's weight is distributed uniformly amongst the datapoints contained within.

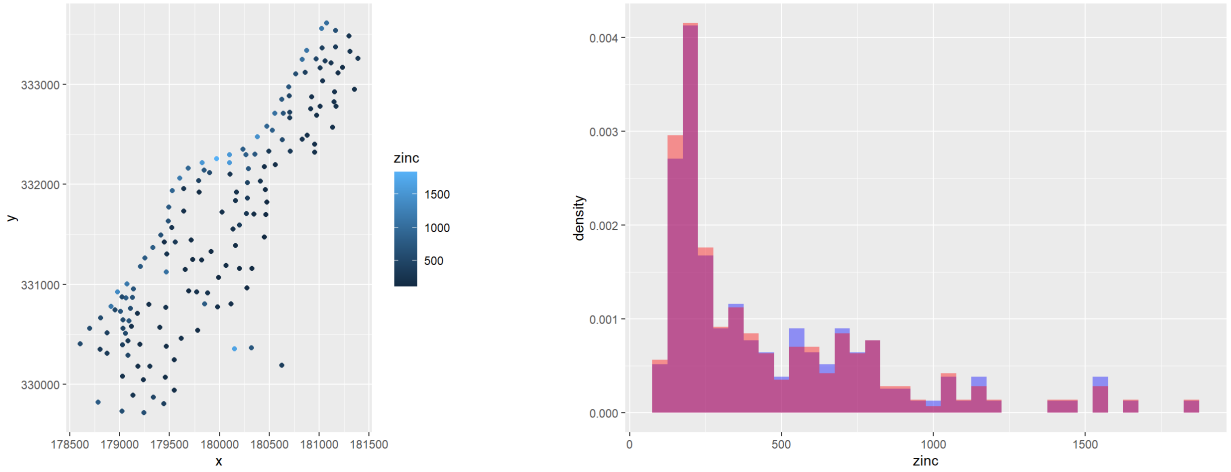
Thus, each datapoint's weight depends on the number of datapoints in its own cell (including itself), as well as the total number of cells that contain data.

These sample weights reduce the influence of regions that are oversampled. In the code below, note that the declustering weights are used to modify the empirical distribution (here, the histogram) – not through scaling the *values* of the variable of interest, but through scaling their *contribution* to the empirical distribution. In `ggplot()`, we achieve this by mapping the `weight` aesthetic in `geom_histogram()` and also mapping `y=after_stat(density)`.

In the following code chunk, we use two specialised functions:

- `raster()` is part of the raster package, which is loaded for the purposes of this example only;
- `SpatialPoints()` is a function that coerces a matrix of coordinates into a specific R class, which interacts well with `raster()`. Raster layers are beyond the scope of this notebook; fully getting to grips with these layers is not crucial to the understanding of the ESDA techniques below. Nevertheless, these functions provide an efficient way of performing cell declustering.

```
library(raster)
ggplot(data = meuse, aes(x = x, y = y)) +  # Initialize the plot, with data and ax
es.
  geom_point(aes(colour = zinc)) +  # Use the point geom, to create a scatter plot
; use colour to encode the zinc levels
  coord_equal()

r <- raster(xmn = min(meuse$x)-100, xmx = max(meuse$x)+100,
            ymn = min(meuse$y)-100, ymx = max(meuse$y)+100,
            res = 100)
xy <- SpatialPoints(cbind(meuse$x,meuse$y))

cellIndices <- cellFromXY(r,xy)
  # cellFromXY() gives the index of the cell that point (x,y) lies in (counting le
ft to right, top to bottom)
counts <- tabulate(cellIndices)
  # this records the number of points in each cell
invCellWts <- sum(counts>0)
  # each cell containing data is equally (uniformly) weighted;
invWts <- invCellWts*counts
  # within each cell, the cell's weight is distributed amongst all samples.
meuse["invWts"] <- invWts[cellIndices]

ggplot(data=meuse, mapping=aes(x=zinc, y=after_stat(density))) +
  geom_histogram(fill='blue', alpha=0.4, binwidth=50) +
  geom_histogram(mapping=aes(weight=1/invWts), fill='red', alpha=0.4, binwidth=50)
```



In the above example, you will note that we can obtain different results from choosing the dimensions of each grid cell; this is controlled by the resolution parameter (`res`) in the call to `raster()` . Increasing this value will result in those values that are potentially over-sampled being downweighted even more; we must be careful, however, to not down-weight them too much. A popular approach to choosing the best cell size uses the sample mean of the variable of interest as a guide: we calculate the declustered mean (that is, the weighted sample mean, using the reciprocal of the cell counts as weights) for a number of different cell sizes, and then use the cell size that results in either:

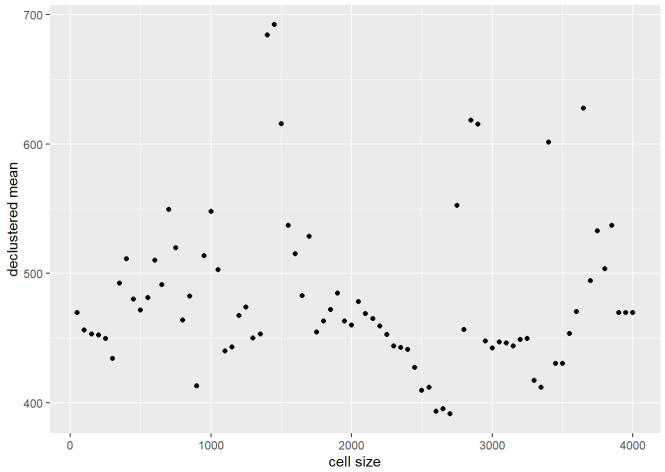
- the lowest declustered mean, if declustering results in lower values of the (weighted) sample mean, i.e. if the random field has been preferentially sampled in regions of high values; or
- the highest declustered mean, if declustering increases the value of the (weighted) sample mean, i.e. if the random field has been preferentially sampled in regions of low values.

Exercises

4. Using the Meuse river dataset, repeat the above cell declustering method, for different values of the cell size; calculate the (declustered) mean zinc concentration in the topsoil.
5. Can we use this declustered mean to choose an appropriate cell size?

We wrap the cell declustering procedure in a function that takes the cell size as an input and outputs the 'declustered mean' of `zinc`. We perform this declustering for a sequence of cell sizes.

```
decluster <- function(cellsize){
  r <- raster(xmn = min(meuse$x)-cellsize, xmx = max(meuse$x)+cellsize,
              ymn = min(meuse$y)-cellsize, ymx = max(meuse$y)+cellsize,
              res = cellsize)
  # Note that we increase the region over which the grid is calculated by extendin
g a distance equal to 'cellsize' in each direction.
  # This is to ensure that all datapoints are included in the resulting grid.
  xy <- SpatialPoints(cbind(meuse$x,meuse$y))

  cellIndices <- cellFromXY(r,xy)
    # cellFromXY() gives the index of the cell that point (x,y) lies in (counting
left to right, top to bottom)
  counts <- tabulate(cellIndices)
    # this records the number of points in each cell
  invCellWts <- sum(counts>0)
    # each cell containing data is equally (uniformly) weighted;
  invWts <- invCellWts*counts
    # within each cell, the cell's weight is distributed amongst all samples.
  meuse["invWts"] <- invWts[cellIndices]

  scaledzinc <- meuse$zinc/meuse$invWts
  return(sum(meuse$zinc/meuse$invWts)) # = sum(Weights * zinc concentrations) / su
m(weights), as sum(weights)=1
}

cellSizes <- seq(50,4000,50)
declusteredMean <- vector(length = length(cellSizes))

i<-0
for (cellSize in cellSizes){
  i <- i+1
  declusteredMean[[i]]<- decluster(cellSize)
}

ggplot(data.frame(x=cellSizes,y=declusteredMean)) +
  geom_point(aes(x=x,y=y)) +
  xlab("cell size") + ylab("declustered mean")
```



In the presence of irregular sampling, the declustered sample mean for a very small cell size (such that each data point is the only data point in its cell) should be equal to the declustered sample mean for a very large cell size (i.e. when all datapoints are in a single cell). In both of these extreme scenarios, the declustered mean is of course equal to the unweighted sample mean. We can see that this behaviour is replicated in the plot above.

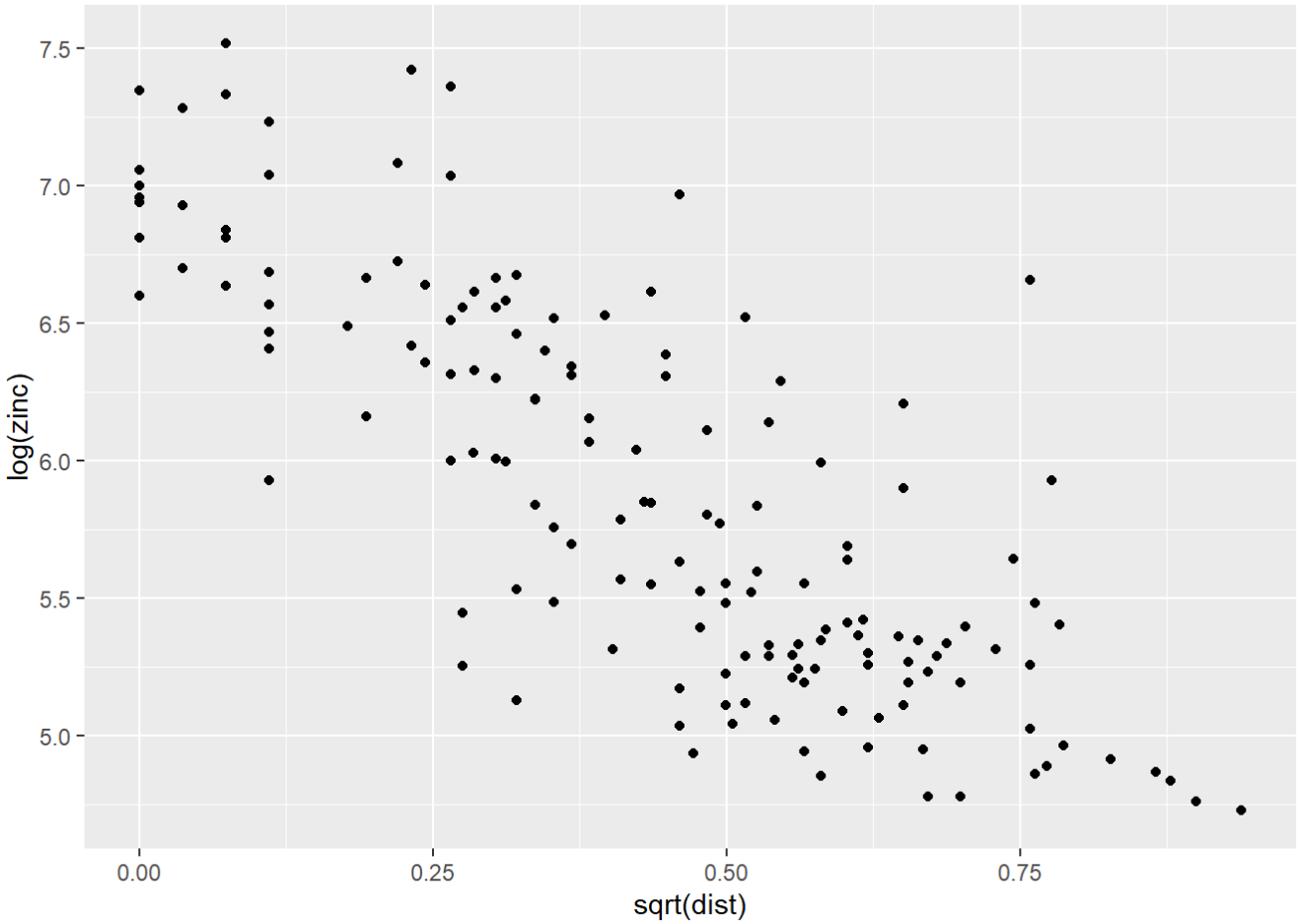
In the presence of *preferential* sampling, however (i.e. where the degree of clustering is related to the value of the variable of interest), I would expect to see the majority of the declustered mean values being greater than (or, alternatively, less than) the unweighted sample mean. The fact that our declustered means fluctuate quite frequently above and below the value of the unweighted mean as we increase the cell size suggests that the spatial clustering does not have a strong dependence with the value of `meuse$zinc`, i.e. that this dataset does not display strong preferential sampling, after all.

Without any strong evidence to perform declustering, we return to the original (unweighted) values of `meuse$zinc`. This data is positively skewed, as we noted before. This would encourage one of two further approaches: we could consider transforming the data (using, for instance, a log-transformation), or we may choose to compare against an alternative theoretical distribution (e.g. the Gamma distribution, which is positively-skewed). As we will see below, a simple log-transformation has some merits for this example.

Identify and fit trend components

Treating our spatial dataset as an observation of a random field, it will once again benefit us to consider whether the underlying process is stationary or nonstationary. We will not consider formal tests of stationarity for spatial data, but it will be useful for us to identify and remove any component of the data that is an obvious source of nonstationarity. We are therefore interested in identifying (and characterising) *trends*.

Identification of any trends can happen as early as the initial mapping stage. For the Meuse river dataset, our initial visualisation of the data showed a clear trend for higher topsoil concentrations of heavy metals, closer to the river. The `meuse` data frame provides a measure of the distance between each observed point and the river bank, and so we can further explore this relationship. Indeed, we find that there is evidence of a linear relationship between `log(meuse$zinc)` and `sqrt(meuse$dist)`.



Exercises

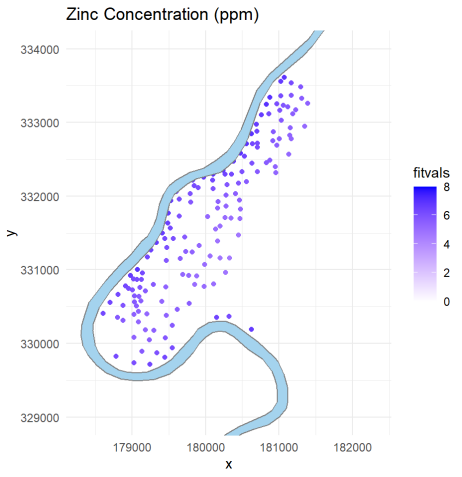
6. Create a model for the trend observed above, by fitting a suitable linear model. Assign the fitted values and residuals to `meuse$fitvals` and `meuse$resids` , respectively.
7. Use `ggplot()` to map both the fitted values for this trend and the residuals.
8. Has this trend fully explained the spatial variation in topsoil concentration of zinc?

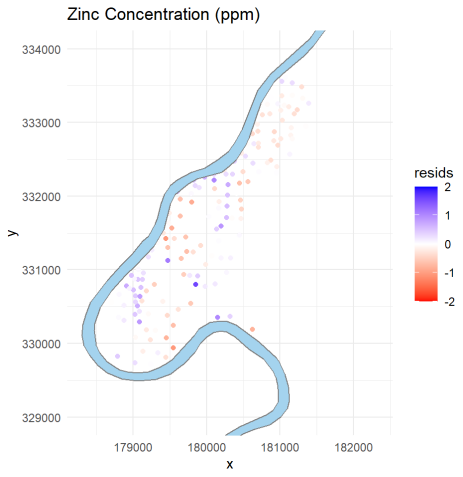
We start by fitting a linear model in the usual way, aiming to predict `log(zinc)` using the values of `sqrt(dist)` as an explanatory variable

```
meuse_lm <- lm(log(zinc)~sqrt(dist), data=meuse)
meuse$fitvals <- meuse_lm$fitted.values
meuse$resids <- meuse_lm$residuals
```

We use a similar ggplot construction as before, to plot the fitted values and the residuals, at the observed spatial locations, for this linear model.

```
# map the fitted values
fittedvals_map <- ggplot(data=meuse, aes(x, y)) + geom_point(aes(colour=fitvals))
+
  scale_colour_gradient(low="white", high="blue", limits=c(0,8)) +
  geom_polygon(data = data.frame(x = meuse.riv[,1],y = meuse.riv[,2]),
               aes(x = x,y = y),
               fill = "lightskyblue2",
               colour= "grey50") +
  ggtitle("Zinc Concentration (ppm)") + coord_equal(ylim=c(329000,334000)) + theme
_bw() +
  scale_size(range = c(0.5, 3.5)) +   theme_minimal()

# map the residuals
residuals_map <- ggplot(data=meuse, aes(x, y)) + geom_point(aes(colour=resids)) +
  scale_colour_gradient2(low="red", mid="white", high="blue", limits=c(-2,2)) +
  geom_polygon(data = data.frame(x = meuse.riv[,1],y = meuse.riv[,2]),
               aes(x = x,y = y),
               fill = "lightskyblue2",
               colour= "grey50") +
  ggtitle("Zinc Concentration (ppm)") + coord_equal(ylim=c(329000,334000)) + theme
_bw() +
  scale_size(range = c(0.5, 3.5)) +   theme_minimal()

fittedvals_map
residuals_map
```





From the above exercises, you should have obtained a visualisation of the residual zinc concentration levels, after this trend has been taken into account. If the zinc concentration was fully explained by this trend component, then these residuals should display complete spatial randomness, i.e. there should be no spatial clustering of similar values. Alas, this is not the case! Your visualisations should show some evidence of a spatial dependence structure within the residuals. We further explore this dependence using notions of spatial autocorrelation.

Examine spatial dependence structures

Consider a variable z in a spatial dataset, and suppose the location information for this dataset is specified by the coordinate vector $\underline{s} = (x, y) \in \mathbb{R}^2$. As noted above, for spatial data, the datapoints observed at $\underline{s}_1, \ldots, \underline{s}_n$ are taken together to be a single observation of an underlying stochastic process; denote this process $\{Z(\underline{s})\}$.

When we wish to estimate the dependence between two random variables, we typically do so using a random sample, containing multiple observations of the same two random variables.

For spatial datasets, however, we cannot typically estimate the spatial dependence in the underlying process – e.g. the correlation between $Z(x_1, y_1)$ and $Z(x_2, y_2)$ – without assuming the underlying process to be stationary. If the process is nonstationary, then the dependence structure will be location-dependent, and of course, we only have access to a single observation of the process at each fixed location! (Note that, of course, the same is true of time series data – we can only estimate the autocovariance sequence $C_{XX}(h)$ once we have assumed the process $\{X_t\}$ to be stationary).

The semivariogram

For spatial processes, it is common to consider an alternative measure of the dependence structure. The semivariogram $\gamma(\underline{h})$ of a spatial process $\{Z(\underline{s})\}$ is closely related to its covariance, and is defined as half the variance of the difference between values of the process at two distinct locations:

$$2\gamma(\underline{h}) = \mathbb{E}\left[(Z(\underline{s}) - Z(\underline{s}+\underline{h}))^2\right].$$

Note that, in order for the variogram to be well-defined, we require a slightly different notion of stationarity: we say that a process is *intrinsically stationary* if the process $Z(\underline{s}) - Z(\underline{s}+h)$ is stationary – this ensures that the semivariogram is a function of $\underline{h}$ only.

The empirical semivariogram

There is an additional problem with irregularly-sampled spatial data, such as the data we are considering here, however. Once we assume stationarity, we have that the spatial dependence between the process at locations $\underline{s}_1$ and $\underline{s}_2$ will be a function of the separation vector $\underline{h} = \underline{s}_2 - \underline{s}_1$. If we further assume *anisotropy*, spatial autocorrelation will be dependent on only the *magnitude* of this separation vector, $\|\underline{h}\|$. For regularly-spaced time series data, we were able to take multiple pairs of datapoints that were separated by the same time lag. Since our spatial data is irregularly spaced, however, it will be unlikely for us to find multiple pairs of observed locations that are separated by the same separation vector $\underline{h}$. In order to *estimate* the spatial dependence structure, we will need to group similar separation vectors (or those with similar magnitudes) together.

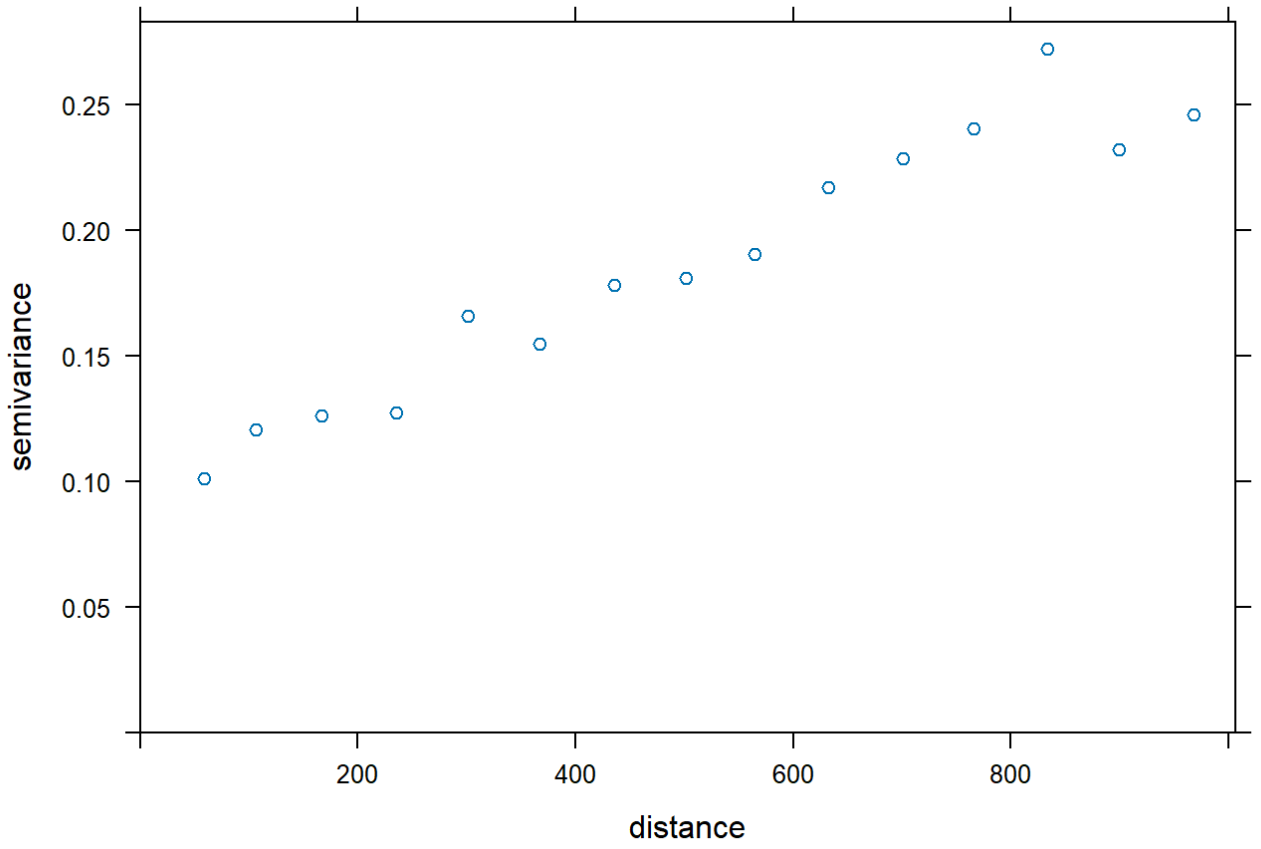
We can estimate the variogram for an anisotropic spatial dataset by choosing a selection of intervals, and calculating a sample-based estimate, using pairs of datapoints that are spatially separated by a distance vector with magnitude in the given interval. So, defining an interval $[h_1, h_2] \subset \mathbb{R}$, we can define the sample semivariogram:

$$\hat{\gamma}(h_1, h_2) = \frac{1}{2N}\sum_{k=1}^{N}\left\{Z(\underline{s}_{i_k}) - Z(\underline{s}_{j_k})\right\}^2,$$

where $\left\{\left(\underline{s}_{i_k}, \underline{s}_{j_k}\right), k = 1, \ldots, N\right\}$ is a collection of N pairs of datapoints, which are each spatially separated by a distance of magnitude $\|\underline{s}_{j_k} - \underline{s}_{i_k}\| \in (h_1, h_2]$. This is the quantity that is estimated by the function `gstat::variogram()`, which is part of the gstat library in R.

In the below code cell, we can demonstrate the use of `gstat::variogram()` for calculating the sample semivariogram for a set of spatial datapoints. Note that in order to do this, we need to first 'promote' the data frame `meuse` into a special class of R object known as a `SpatialPointsDataFrame`. Note also that you will need to uncomment a couple of lines of code, which depend on the variables assigned in Exercise 6.

```
# the following line coerces the data frame `meuse` into a 'SpatialPointsDataFrame
' by specifying that the variables x and y (to be found in meuse) are the coordina
tes.
coordinates(meuse) <- ~x+y
# meuse is now a SpatialPointsDataFrame...
meuse_var <- variogram(resids~1, meuse, cutoff=1000)
plot(meuse_var)
```



An alternative empirical approach

There are some alternative approaches to quantifying spatial dependence, which are suitable for ESDA.

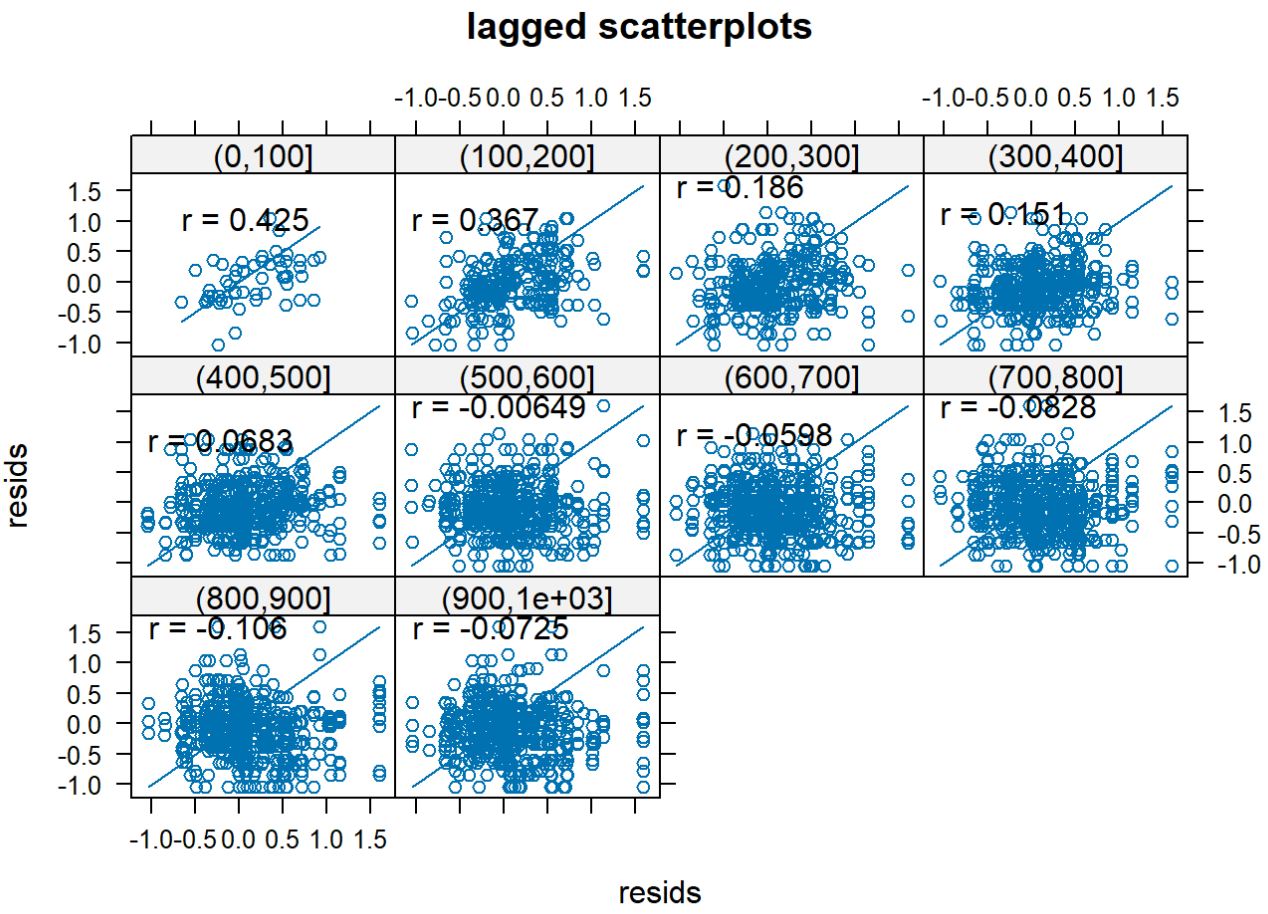
One simple way to identify whether spatial correlation is present, is to create scatter plots of datapoint pairs $z(\underline{s}_i)$ and $z(\underline{s}_j)$, grouped according to the separation distances $h_{ij} = \|\underline{s}_i - \underline{s}_j\|$. This is implemented by the `gstat::hscat()` function in R; this function plots each group in a separate panel, and reports the sample correlation for each group.

Exercises

Again considering the `meuse` dataset:

9. Use the `hscat()` function to assess whether there is any spatial autocorrelation present in the residual zinc concentration levels, which you obtained after accounting for distance from the river.

```
hscat(resids~1, meuse, breaks=seq(0,1000,100))
```



`hscat()` works by considering each of the separation 'bins' specified by the `breaks` parameter separately. In this example, for instance, we can see that there is a small degree of positive correlation (r=0.425) between pairs of residual values, at locations separated by $h_{ij} \in (0, 100]$. As the separation increases, this correlation noticably diminishes, indicating a 'small scale' spatial autocorrelation: positive spatial dependence in the residuals is greater when measured over shorter distances.

Using ggplot2 to create a choropleth map

As described at the top of this notebook, the spatial component of a set of data may be provided by a nominal variable, such as the name of a geographical region. We can visualise such data using the ggplot2 library; in particular, we will make use of the 'polygon' geom.

Our first aim is to use `geom_polygon()` to plot the outlines of a geographical region. We can do so by making use of the function `ggplot2::map_data()`. This will extract coordinate data from the `maps` library, to create a data frame containing the boundaries of one of a selection of geographical regions. For instance, `map_data("state")` will provide a dataframe containing the boundaries of all of the states on the US mainland, generated from US Department of the Census data.

Once we have the coordinates for the boundaries of our spatial regions, we can match this to the values of our spatial variable of interest using one of the 'mutating joins' from the dplyr library; these allow us to add columns from one data frame to another, matching rows based on the 'location' variable.

In the following code cell, we demonstrate these steps. We obtain the US state boundaries as described above, and we combine this with state-level life expectancy values (which is contained in the data frame `state.x77`). We then createa a choropleth map using `geom_polygon()`, encoding the life expectancy with the colour aesthetic.

```
library(ggplot2)
library(dplyr)
```

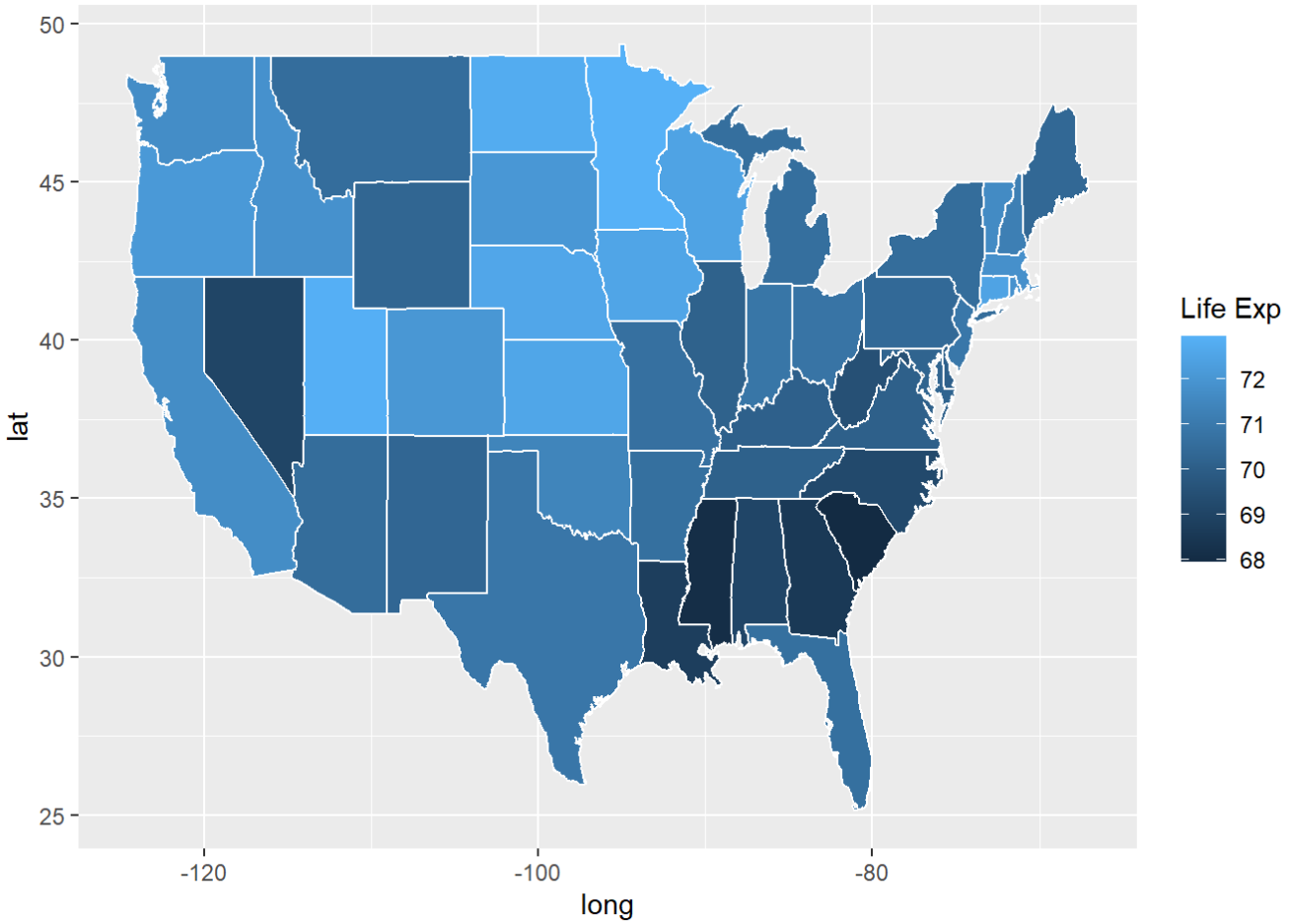
```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:raster':
##
##     intersect, select, union
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
states_map <- map_data("state")

states_data<-as.data.frame(state.x77)

states_data$region <- tolower(rownames(states_data))

fact_join <- left_join(states_map, states_data, by = "region")


ggplot(fact_join, aes(long, lat, group = group))+
  geom_polygon(aes(fill = `Life Exp`), color = "white")
```



References

1. Bivand, R.S., Pebesma, E.J. and Gomez-Rubio, V., 2013. Applied Spatial Data Analysis with R (2nd Ed.) Chapter 8.